

Computer Vision: Task 1 Report

Image Processing Algorithms & UI Integration

Alaa Abdelnasser Alaa Mubarak Basma Mohamed
Rehab Marzouk Lyan Ahmed

Contents

1	Introduction	2
2	Additive Noise Generation	2
2.1	Algorithms & Implementation	2
2.1.1	Uniform Noise	2
2.1.2	Gaussian Noise	2
2.1.3	Salt & Pepper Noise	2
2.2	UI Integration & Results	3
2.3	Observations on Parameter Variations	4
3	Low-Pass Filtering (Spatial Domain)	5
3.1	Algorithms & Implementation	5
3.2	UI Integration & Results	5
3.3	Observations on Parameter Variations	6
4	Edge Detection	6
4.1	Algorithms & Implementation	6
4.2	UI Integration & Results	7
4.3	Observations on Operator Variations	7
5	Histograms & Cumulative Distribution Function (CDF)	8
5.1	Algorithms & Implementation	8
5.2	UI Integration & Results	8
5.3	Observations	9
6	Contrast Enhancement	10
6.1	Algorithms & Implementation	10
6.2	UI Integration & Results	10
6.3	Observations on Enhancement Techniques	11
7	Frequency Domain Filtering & Hybrid Images	11
7.1	Algorithms & Implementation	11
7.2	UI Integration & Results	12
7.3	Observations	12
8	Conclusion	13

1 Introduction

This report documents the implementation of a comprehensive desktop application built using C++ and the **Qt Framework**, integrated with **OpenCV** for core image processing algorithms. The application serves as a toolkit for fundamental computer vision tasks operating in both the spatial and frequency domains.

The report evaluates the implementation details, UI integration, and visual observations of various algorithms including additive noise generation, low-pass/high-pass filtering, edge detection, histogram equalization, and the creation of hybrid images.

2 Additive Noise Generation

The objective of this task is to implement and apply three different types of additive noise to an image: Uniform, Gaussian, and Salt & Pepper noise. The application allows users to dynamically adjust the parameters for each noise type via the GUI.

2.1 Algorithms & Implementation

The noise generation is handled within the `onApplyNoiseClicked()` function. Based on the user's selection from the combo box, the appropriate mathematical model is applied.

2.1.1 Uniform Noise

Uniform noise adds random values to the image pixels, where every value in a specified range $[min, max]$ has an equal probability of occurring. We used OpenCV's `cv::randu` to generate a noise matrix bounded by the user-defined limits.

```
1 // Uniform Noise: Additive random values bounded by min/max
2 cv::randu(noiseImg, cv::Scalar::all(spinUniformMin->value()), cv::
   Scalar::all(spinUniformMax->value()));
3 cv::add(processedImage, noiseImg, result, cv::noArray(), processedImage
   .type());
```

2.1.2 Gaussian Noise

Gaussian noise models natural sensor noise, following a normal distribution. We used `cv::randn` to generate this noise. The algorithm calculates the standard deviation (σ) mathematically as $\sigma = \sqrt{variance}$ to feed it into the function.

```
1 // Gaussian Noise: Normal Distribution (Mean, StdDev)
2 cv::randn(noiseImg, cv::Scalar::all(spinGaussMean->value()), cv::Scalar
   ::all(std::sqrt(spinGaussVar->value())));
3 cv::add(processedImage, noiseImg, result, cv::noArray(), processedImage
   .type());
```

2.1.3 Salt & Pepper Noise

This noise simulates dead pixels by randomly turning pixels completely black (Pepper) or completely white (Salt). A probability matrix is generated using `cv::randu(0.0, 1.0)`. For a given probability p , a pixel becomes black if the random value is $< p/2$, and white if $> 1 - (p/2)$.

```

1 // Probability matrix generation
2 cv::randu(randMat, 0.0, 1.0);
3 for (int i = 0; i < result.rows; i++) {
4     for (int j = 0; j < result.cols; j++) {
5         float randomValue = randMat.at<float>(i, j);
6         if (randomValue < prob / 2.0) { // Pepper (Black)
7             if (result.channels() == 3) result.at<cv::Vec3b>(i, j) = cv
::Vec3b(0, 0, 0);
8             else result.at<uchar>(i, j) = 0;
9         } else if (randomValue > 1.0 - (prob / 2.0)) { // Salt (White)
10             if (result.channels() == 3) result.at<cv::Vec3b>(i, j) = cv
::Vec3b(255, 255, 255);
11             else result.at<uchar>(i, j) = 255;
12         }
13     }
14 }

```

2.2 UI Integration & Results

The application interface utilizes a combo box to select the noise type and dynamically displays the corresponding parameter inputs. Below are the results of applying each noise model.

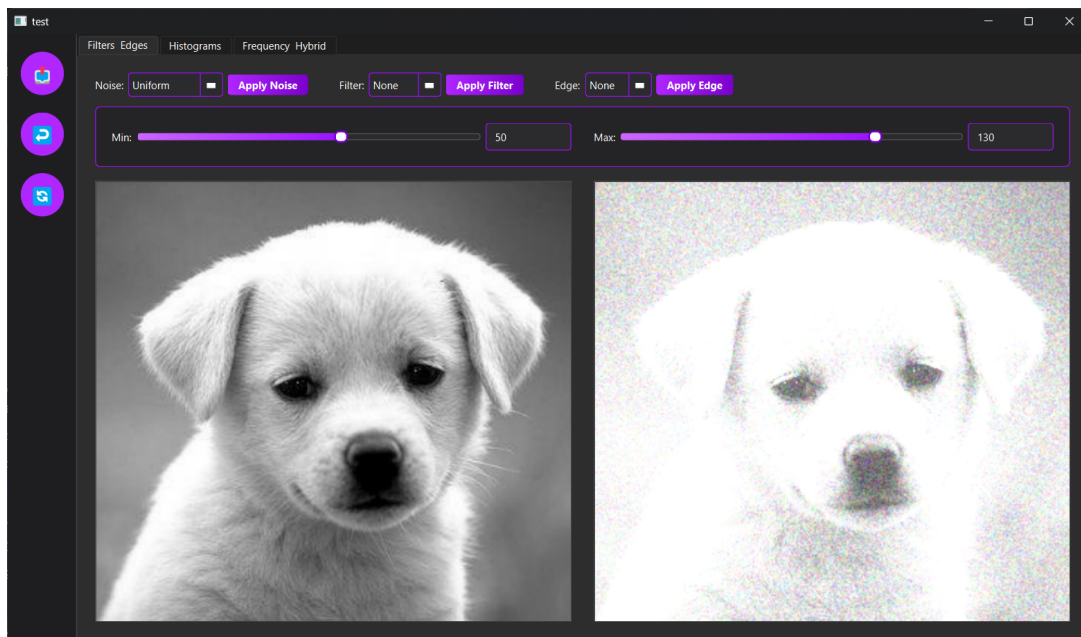


Figure 1: Application of Uniform Noise (Min = 50, Max = 130).

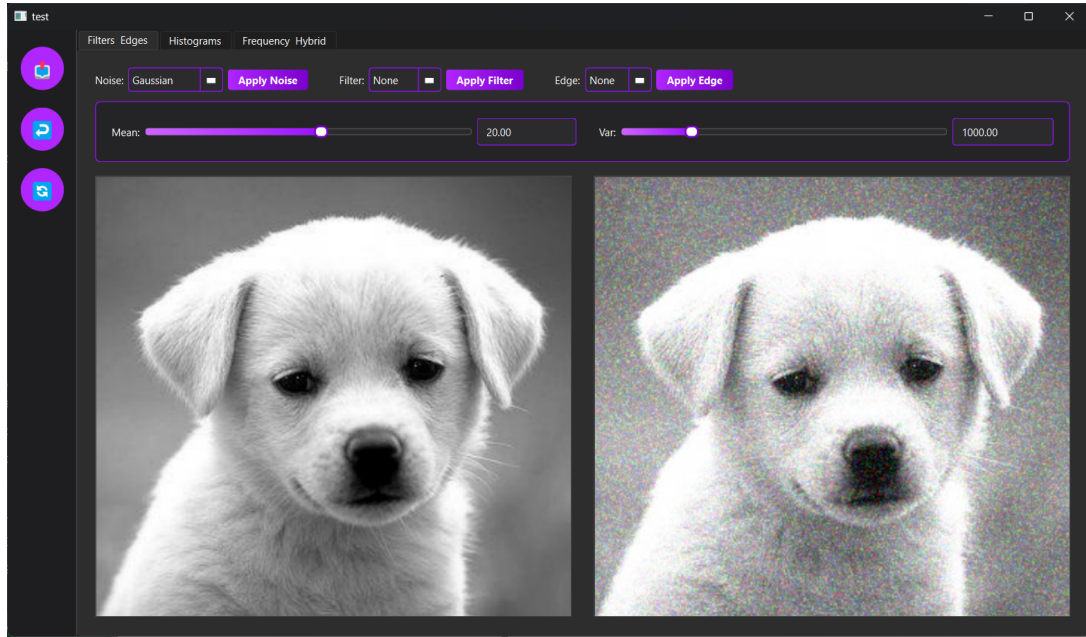


Figure 2: Application of Gaussian Noise ($\mu = 20, \sigma^2 = 1000$).

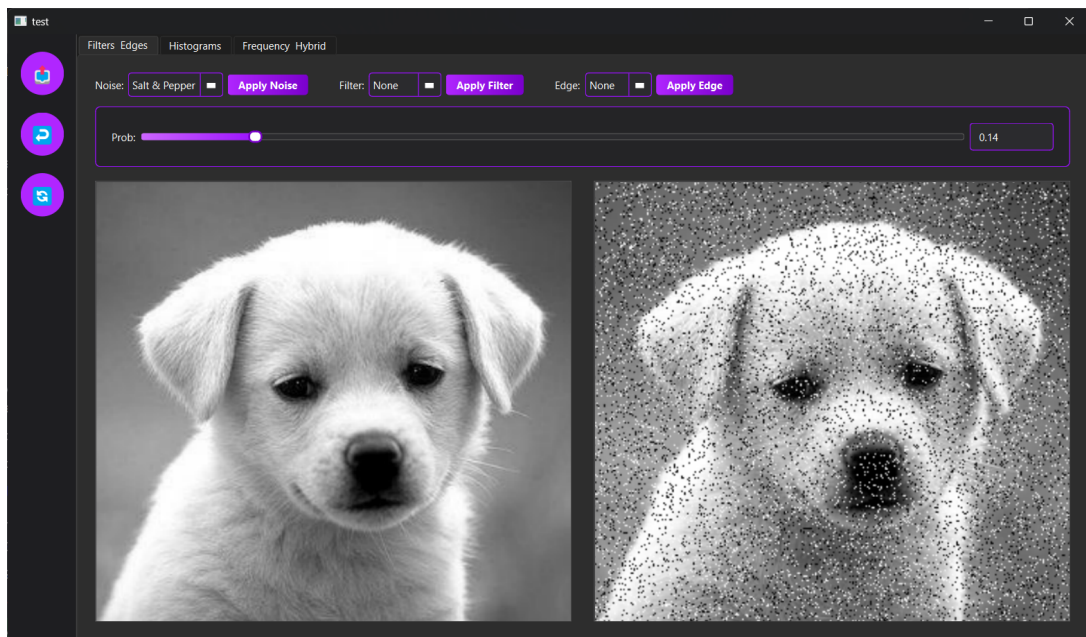


Figure 3: Application of Salt & Pepper Noise (Probability $p = 0.14$).

2.3 Observations on Parameter Variations

- **Uniform & Gaussian Noise:** Expanding the range/variance significantly degrades the image details, making the image appear heavily "grainy" and reducing contrast.
- **Salt & Pepper Noise:** The parameter p directly correlates to the percentage of corrupted pixels. At higher probabilities (e.g., $p > 0.4$), structural details become almost entirely lost.

3 Low-Pass Filtering (Spatial Domain)

The objective of this task is to apply spatial low-pass filters to reduce image noise. The application supports Average, Gaussian, and Median filters with user-selectable kernel sizes (3×3 and 5×5).

3.1 Algorithms & Implementation

Based on the user's GUI selection, the kernel size is extracted, and the appropriate OpenCV filtering operation is applied:

- **Average Filter:** Computes the mean of the kernel neighborhood.
- **Gaussian Filter:** Applies a weighted average based on a Gaussian distribution, preserving edges slightly better than the average filter.
- **Median Filter:** A non-linear filter that replaces the pixel with the median value of its neighborhood.

```
1 int ksize = (comboMask->currentIndex() == 0) ? 3 : 5; // Extract Kernel Size
2 cv::Mat result;
3
4 // Apply specific spatial filtering using OpenCV built-in functions
5 if (type == 1) cv::blur(processedImage, result, cv::Size(ksize, ksize));
6 else if (type == 2) cv::GaussianBlur(processedImage, result, cv::Size(ksize, ksize), 0);
7 else if (type == 3) cv::medianBlur(processedImage, result, ksize);
```

3.2 UI Integration & Results

To evaluate the effectiveness of each filter, we applied them directly to the noisy images generated in the previous section using a 5×5 kernel size. The Average filter was applied to the Uniform noise, the Gaussian filter to the Gaussian noise, and the Median filter to the Salt & Pepper noise.



(a) Average Filter
(on Figure 1)

(b) Gaussian Filter
(on Figure 2)

(c) Median Filter
(on Figure 3)

Figure 4: Application of 5×5 filters targeting specific noise types.

3.3 Observations on Parameter Variations

- **Filter Type Suitability:** As observed, the Median filter is exceptionally effective at completely removing Salt & Pepper noise (Figure 4c) without blurring the edges, as it ignores extreme outlier values. The Average and Gaussian filters successfully smooth out Uniform and Gaussian noise but at the cost of noticeable image blurring.
- **Kernel Size (3×3 vs. 5×5):** Increasing the kernel size to 5×5 increases the noise reduction capability, but for Average and Gaussian filters, it causes a much more pronounced loss of fine details.

4 Edge Detection

The objective of this task is to implement and compare different edge detection operators. The application evaluates first-order derivative masks (Sobel, Prewitt, Roberts) implemented manually via 2D convolution, alongside the multi-stage Canny edge detector.

4.1 Algorithms & Implementation

Prior to edge detection, the image is converted to Grayscale.

- **Sobel & Prewitt:** Utilize 3×3 kernels to compute the gradient approximations in the horizontal (G_x) and vertical (G_y) directions. Sobel applies a heavier weight to the center pixels, providing slight noise reduction compared to Prewitt.
- **Roberts Cross:** Uses a minimal 2×2 kernel to compute diagonal gradients.
- **Magnitude Calculation:** For the manual implementations, the final edge magnitude is computed mathematically as $M = \sqrt{G_x^2 + G_y^2}$.
- **Canny:** Implemented via OpenCV due to its multi-stage pipeline (Gaussian smoothing, gradient calculation, non-maximum suppression, and hysteresis thresholding).

To enforce DRY (Don't Repeat Yourself) architecture, a unified convolution loop evaluates both 2×2 and 3×3 kernels dynamically:

```
1 // Unified Convolution Loop for Roberts (2x2) or Sobel/Prewitt (3x3)
2 for (int i = offset; i < gray.rows - 1; i++) {
3     for (int j = offset; j < gray.cols - 1; j++) {
4         int gx = 0, gy = 0;
5         if (isRoberts) { // Roberts Cross Gradient Operators
6             gx = gray.at<uchar>(i, j) - gray.at<uchar>(i+1, j+1);
7             gy = gray.at<uchar>(i, j+1) - gray.at<uchar>(i+1, j);
8         } else { // Standard 3x3 Convolution
9             for (int ki = -1; ki <= 1; ki++) {
10                 for (int kj = -1; kj <= 1; kj++) {
11                     int pixel = gray.at<uchar>(i + ki, j + kj);
12                     gx += pixel * kx[ki + 1][kj + 1];
13                     gy += pixel * ky[ki + 1][kj + 1];
14                 }
15             }
16         }
17         // Calculate gradient magnitude: M = sqrt(gx^2 + gy^2)
18         result.at<uchar>(i, j) = cv::saturate_cast<uchar>(std::sqrt(gx*gx + gy*gy));
```


4.2 UI Integration & Results

The following figure demonstrates the application of the four edge detection operators on the same input image for a direct visual comparison.



(a) Sobel Operator



(b) Prewitt Operator



(c) Roberts Cross



(d) Canny Edge Detector

Figure 5: Comparison of different edge detection algorithms.

4.3 Observations on Operator Variations

- **Sobel vs. Prewitt:** Both yield similar results with relatively thick edges. Sobel produces slightly smoother edges due to the weighted center in its kernel, making it slightly more robust to noise than Prewitt.
- **Roberts:** Generates finer and thinner edges due to the smaller 2×2 mask, but it is highly sensitive to noise and often fails to capture weaker edges.

- **Canny:** Clearly provides the superior result. The edges are crisp, strictly one-pixel thin (due to non-maximum suppression), and continuous (due to hysteresis thresholding).

5 Histograms & Cumulative Distribution Function (CDF)

The objective of this task is to compute and visualize the intensity distribution of an image via its Histogram, alongside its Cumulative Distribution Function (CDF). The application dynamically supports both Grayscale (single channel) and RGB (three channels) images.

5.1 Algorithms & Implementation

- **Histogram:** Represents the frequency of each intensity value $[0, 255]$ in the image. We utilized `cv::calcHist` to extract these frequencies.
- **CDF:** Represents the cumulative sum of the histogram bins up to a certain intensity level. Mathematically, it is defined as: $CDF(v) = \sum_{i=0}^v h(i)$, where $h(i)$ is the histogram value at intensity i .
- **RGB Processing:** For color images, the image is split into its separate Blue, Green, and Red channels using `cv::split`, and the Histogram/CDF is computed and drawn for each channel independently.

The following snippet demonstrates the core computation logic for a given channel:

```

1 // Calculate histogram (counts intensity frequency)
2 cv::calcHist(&channelImg, 1, 0, cv::Mat(), hist, 1, &histSize,
   histRange);
3
4 // Compute CDF (Cumulative Distribution Function) iteratively
5 cdf = hist.clone();
6 for (int i = 1; i < histSize; i++) {
7     cdf.at<float>(i) += cdf.at<float>(i - 1); // Accumulate previous
   values
8 }

```

5.2 UI Integration & Results

The application features a dedicated tab for Histogram and CDF analysis. The UI automatically adapts based on the selected mode (Grayscale vs. RGB), plotting the data using appropriate color coding (Gray for Grayscale, and Blue/Green/Red for RGB channels).

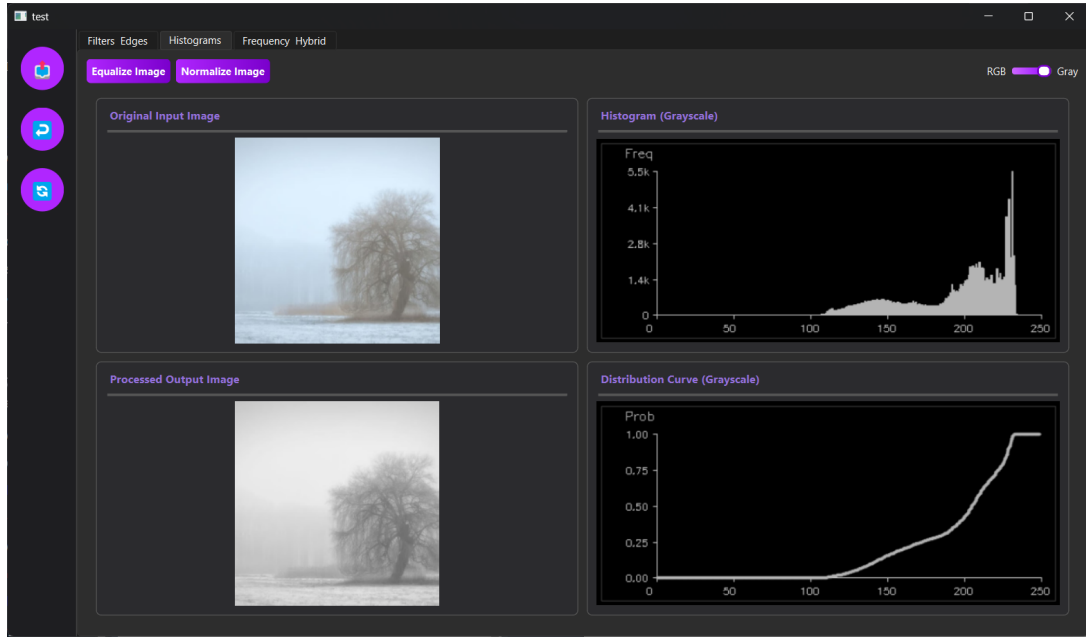


Figure 6: Histogram and CDF visualization for a Grayscale image.

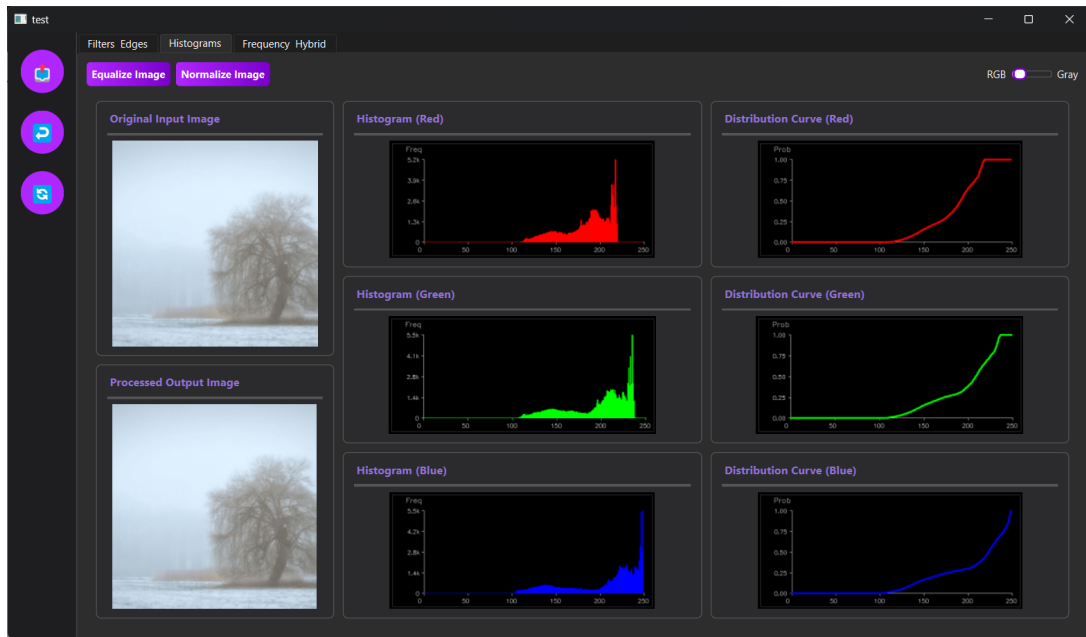


Figure 7: Independent Histogram and CDF visualizations for R, G, and B channels.

5.3 Observations

- **Histogram Analysis:** The spread of the histogram indicates the image contrast. A narrow histogram implies low contrast, while a widely spread histogram implies high contrast. Peaks in the RGB histograms show the dominant colors in the image.
- **CDF Characteristics:** As expected mathematically, all generated CDF curves are monotonically increasing, starting from the lowest present intensity and reaching their maximum at the highest present intensity.

6 Contrast Enhancement

The objective of this task is to improve the visual quality of low-contrast images using two different spatial domain techniques: Min-Max Normalization (Linear Contrast Stretching) and Histogram Equalization (Non-linear Enhancement).

6.1 Algorithms & Implementation

- **Normalization:** Performs a linear transformation on the pixel values, stretching the minimum and maximum intensity values to span the full dynamic range $[0, 255]$ using `cv::normalize` with `cv::NORM_MINMAX`.
- **Histogram Equalization:** A non-linear process that redistributes intensity values so that the resulting histogram is approximately flat (and the CDF is linear), enhancing the global contrast.
- **Color Preservation:** Applying equalization directly to RGB channels causes severe color shifts. To prevent this, color images are first converted to the **YCrCb** color space. Equalization is applied *only* to the Y channel (Luminance/Brightness), leaving the color information (Cr, Cb) intact, before converting back to BGR.

```
1 // 1. Normalization (Linear Contrast Stretching)
2 cv::normalize(processedImage, processedImage, 0, 255, cv::NORM_MINMAX);
3
4 // 2. Color Image Equalization (Preserving Hue/Saturation)
5 cv::cvtColor(processedImage, ycrb, cv::COLOR_BGR2YCrCb);
6 cv::split(ycrb, channels);
7 // Apply equalization ONLY on the Y channel (Luminance)
8 cv::equalizeHist(channels[0], channels[0]);
9 cv::merge(channels, ycrb);
10 cv::cvtColor(ycrb, processedImage, cv::COLOR_YCrCb2BGR);
```

6.2 UI Integration & Results

The application provides direct buttons to apply either Normalization or Equalization. The UI automatically updates both the image and its corresponding histogram/CDF plots to reflect the new intensity distributions.

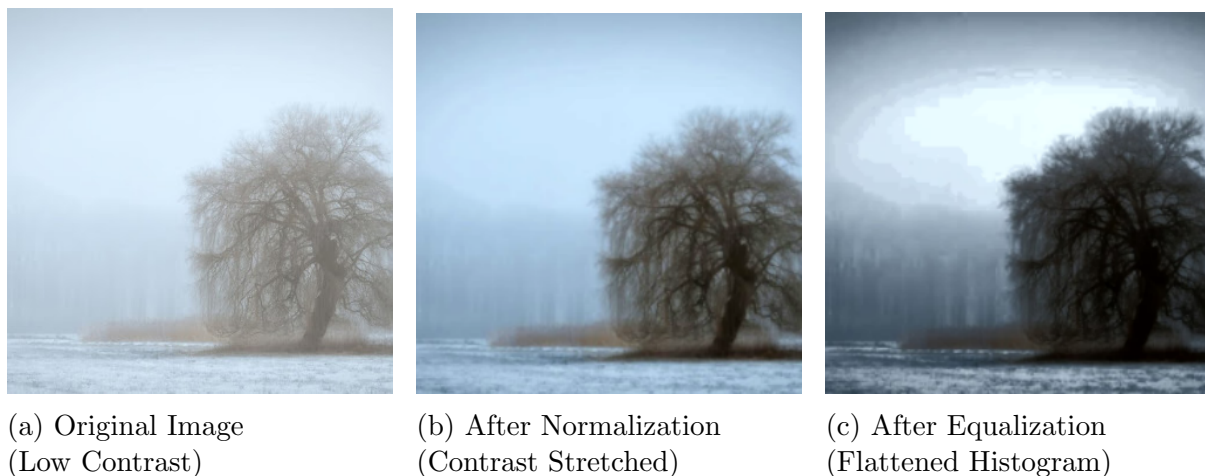


Figure 8: Comparison of contrast enhancement techniques.

6.3 Observations on Enhancement Techniques

- **Normalization:** It perfectly preserves the relative shape of the original histogram but stretches it over the entire $[0, 255]$ range. The result looks natural and removes any "washed out" or faded appearance without altering the relationship between pixel intensities.
- **Equalization:** It drastically changes the shape of the histogram, pushing it to be uniformly distributed. While it significantly increases contrast (often revealing hidden details in dark or overly bright areas), it can sometimes over-enhance noise or produce a harsh, unnatural look compared to simple normalization.

7 Frequency Domain Filtering & Hybrid Images

The objective of this task is to create a "Hybrid Image" by combining the low-frequency components of one image with the high-frequency components of another. This requires transforming the images into the frequency domain using the Discrete Fourier Transform (DFT).

7.1 Algorithms & Implementation

The core processing is handled via a reusable lambda function that applies frequency filters:

- **Transform:** Images are resized and padded to the optimal DFT size, then transformed using `cv::dft`. The zero-frequency component (DC) is shifted to the center of the spectrum.
- **Filtering (Masking):** We implemented Ideal Low-Pass and High-Pass filters. The cutoff decision is based on the Euclidean distance D from the center (c_y, c_x) :
$$D(i, j) = \sqrt{(i - c_y)^2 + (j - c_x)^2}$$
- **Inverse Transform:** The filtered complex spectrum is shifted back, and the Inverse DFT (`cv::idft`) brings the image back to the spatial domain.
- **Combination:** The resulting low-pass and high-pass spatial images are averaged and normalized to produce the final hybrid image.

```
1 // Shifting zero-frequency to the center
2 q0.copyTo(tmp); q3.copyTo(q0); tmp.copyTo(q3);
3 q1.copyTo(tmp); q2.copyTo(q1); tmp.copyTo(q2);
4
5 // Ideal Filter Mask Generation
6 float dist = std::sqrt(std::pow(i - cy, 2) + std::pow(j - cx, 2));
7 if (isHighPass) {
8     mask.at<float>(i, j) = (dist > radius) ? 1.0f : 0.0f; // High-Pass
9 } else {
10     mask.at<float>(i, j) = (dist <= radius) ? 1.0f : 0.0f; // Low-Pass
11 }
12
13 // Apply Mask and Inverse DFT
14 cv::multiply(complexImg, complexMask, complexImg);
15 cv::idft(complexImg, complexImg);
```

7.2 UI Integration & Results

A key feature of the application's UI is real-time visual feedback. As the user adjusts the cutoff frequency sliders, the application dynamically updates and displays three distinct views simultaneously:

1. The low-pass filtered version of the first image.
2. The high-pass filtered version of the second image.
3. The final reconstructed hybrid image.

This allows for precise tuning of the frequency radii before finalizing the merge. Figure 9 demonstrates the complete application interface showcasing this real-time update capability.

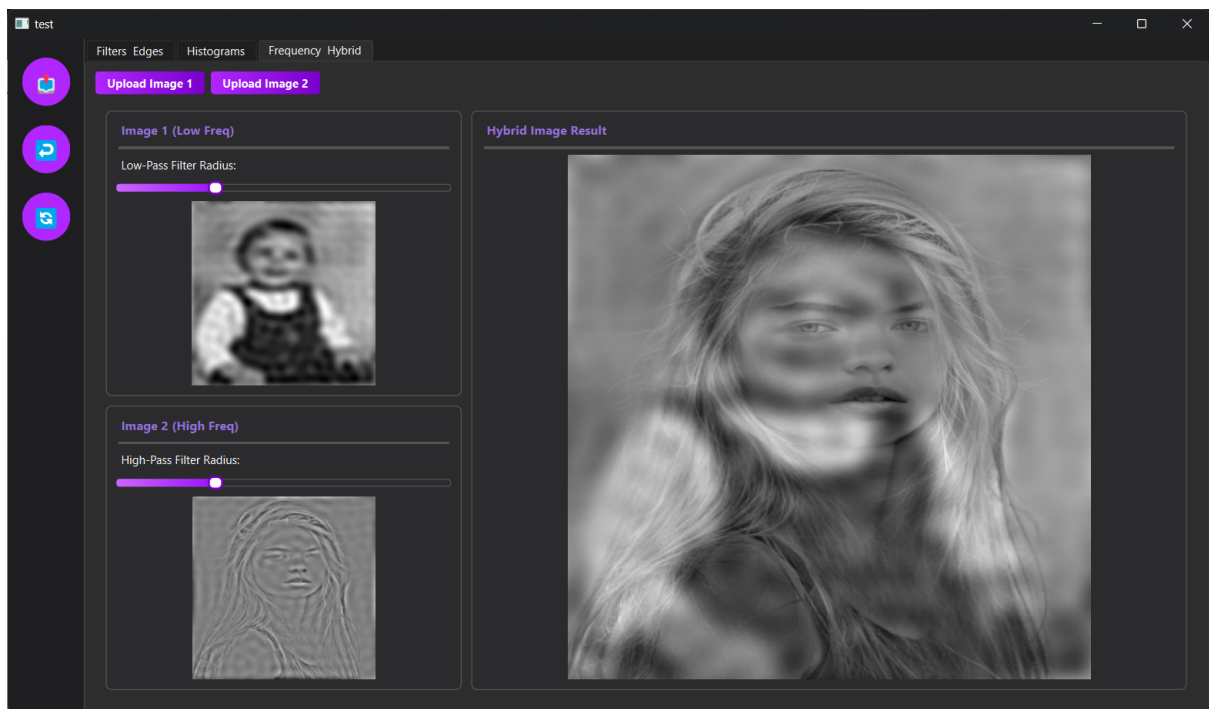


Figure 9: The application interface displaying real-time updates of the Low-Pass, High-Pass, and final Hybrid images.

7.3 Observations

- **Distance Perception:** The hybrid image acts as an optical illusion. When viewed from up close, the sharp edges (high frequencies) from Image 2 dominate our perception. When viewed from a distance, the blurred colors and broad shapes (low frequencies) from Image 1 become dominant.
- **Cutoff Frequencies:** Tuning the sliders is critical. If the low-pass radius is too large, Image 1 overpowers Image 2. If the high-pass radius is too small, Image 2 contributes too much clutter.

8 Conclusion

In conclusion, this project successfully developed a comprehensive, interactive desktop application for fundamental computer vision operations. By leveraging the Qt framework for the graphical interface and OpenCV for mathematical and matrix operations, we built a tool capable of efficiently handling both spatial and frequency domain transformations. The real-time visual feedback integrated into the UI significantly enhances the user experience, allowing for an intuitive understanding of complex algorithmic behaviors, such as frequency cutoff tuning in hybrid images or intensity redistribution in histogram equalization. Ultimately, this implementation provided a robust practical foundation for understanding low-level image processing techniques.